

AI Test Lab and MOSAIC: A Multi-Agent Approach to Faster, More Actionable Software Testing

Research-style academic document prepared for project presentation and professor review

Project: AI Test Lab

Framework: MOSAIC (Multi-agent Orchestrated Software Analysis with Inter-agent Consensus)

Date: April 23, 2026

Abstract

Software testing is one of the most important activities in software engineering because it reduces risk, improves reliability, and helps teams identify defects before deployment. In practice, however, modern testing is often fragmented across many tools and roles: developers write unit tests, quality teams run end-to-end scenarios, CI pipelines execute automated checks, and security teams add scanners or manual reviews. This fragmentation creates overhead in triage, explanation, test writing, patch preparation, and pull request packaging. AI Test Lab addresses this workflow problem by combining repository analysis, specialist-agent review, verifier-backed defect assessment, generated regression tests, proposed code fixes, and draft pull request preparation in a single interface. Its MOSAIC pipeline uses multiple specialist agents for distinct perspectives such as unit logic, black-box behavior, edge cases, security, white-box path reasoning, and performance, then applies a verifier and consensus stage before recommending code changes. The central claim of this document is not that AI Test Lab replaces conventional testing, but that it compresses the first-pass debugging and test-authoring loop into a faster, more explainable workflow. In that sense, the system is designed to save developer time by reducing context switching and manual synthesis while preserving safety through confidence thresholds and patch gating.

Keywords: software testing, multi-agent systems, AI-assisted testing, regression testing, code review, developer productivity, pull request automation

1. Introduction

Software testing exists because software fails in ways that are expensive, risky, and difficult to diagnose after release. Even when teams follow good engineering practices, defects can still appear in logic, edge-case handling, security, and performance. Standards bodies treat testing as a structured engineering activity rather than an optional cleanup step. ISO/IEC/IEEE 29119 describes software testing as a process that applies across manual and automated work, dynamic testing, functional and non-functional testing, and multiple lifecycle models, while emphasizing a risk-based approach to prioritization [1]. In other words, testing is not just about running scripts; it is about deciding what matters most, validating those assumptions, and using evidence to reduce delivery risk.

In ordinary student and industry projects, the practical workflow is usually more chaotic than the standards suggest. Teams often use separate tools for unit tests, CI, static scanning, end-to-end checks, and pull request review. The result is that finding an issue is only the beginning. Someone still has to answer at least five follow-up questions: What exactly is wrong? Where is it? How serious is it? What test should be written? How should the fix be proposed and communicated? AI Test Lab was built around this gap. Instead of treating “testing” as only one command in CI, it treats testing as a broader decision-support process that includes defect identification, explanation, prioritization, regression prevention, and change packaging.

2. What Software Testing Is

At its simplest, software testing is the process of checking whether software behaves as expected under selected conditions. In practice, that simple definition expands into multiple testing levels and perspectives. Unit testing focuses on individual functions or classes. Component and integration testing examine how modules interact. End-to-end testing evaluates full workflows as a user would experience them. Security testing looks for weaknesses such as injection, exposure, broken validation, or unsafe defaults. Performance testing asks whether the code still behaves acceptably as workload or complexity increases.

The literature shows that even something as seemingly straightforward as unit testing is interpreted differently across organizations. Runeson’s survey of 19 companies found that there was broad agreement on the scope of unit testing, but important variation in goals and practices [2]. This matters because it shows that testing is not only a technical activity; it is also a workflow and quality-management choice. Teams may agree that testing is important, while still differing on who writes tests, how much isolation is needed, when tests run, and what evidence is considered sufficient.

Core purposes of testing

- Detect defects before release
- Prevent regressions during code changes
- Validate intended behavior and requirements
- Reduce risk in security, correctness, and performance
- Increase confidence before merge or deployment

Why testing still feels hard

- It spans many tools and test levels
- Failures can be noisy, flaky, or poorly explained
- Coverage does not automatically mean correctness
- Good tests still require domain understanding
- Finding an issue is easier than fixing and packaging it

3. How People Commonly Do Testing Today

Current software teams typically combine several testing modes rather than relying on one. Frameworks such as `pytest` organize developer-written unit and functional tests, along with fixtures, assertions, and test selection conventions [3]. Continuous integration systems such as GitHub Actions run builds and tests on each push or pull request, which helps teams detect errors earlier and spend less time debugging large batches of changes [4]. Static analysis and code scanning add another layer by searching for vulnerabilities and coding errors before or alongside execution [5]. Security guidance from OWASP recommends using both automated security testing and manual expert testing, with automation embedded into build and deployment processes wherever possible [6]. For web applications, end-to-end and component testing tools such as Cypress help teams validate user-visible workflows and isolate component behavior, while also acknowledging that these tests are more difficult to set up and maintain than smaller isolated tests [7].

This tool ecosystem is powerful, but it has a coordination cost. A typical debugging cycle may require a developer to read CI output, open failing files, infer the real root cause, decide whether the problem is logic, edge-case handling, security, or performance, write or improve regression tests, modify code, rerun checks, and then write a pull request summary. Each step may use a different tool and a different mental model. The gap that

AI Test Lab tries to close is therefore not “testing does not exist,” but rather “testing evidence is scattered and expensive to turn into action.”

Current testing practice	Typical benefit	Typical friction
Unit tests with frameworks such as <code>pytest</code>	Fast feedback for isolated logic	Must still be written, maintained, and interpreted manually
CI workflows with GitHub Actions	Automatic execution on push and pull request	Logs can be noisy; failures still need human triage
Static and security scanning	Catches risks early without executing all code paths	False positives and prioritization overhead
End-to-end testing	Validates user-facing workflows	Heavier setup, longer runs, harder maintenance
Manual code review and debugging	Context-aware judgment	Slow, person-dependent, and not easily scalable

Table 1. Common testing practices are valuable individually, but they often leave teams with fragmented evidence and manual follow-up work.

4. AI-Assisted Testing Before AI Test Lab

Recent work has shown that large language models can contribute meaningfully to test generation and software development, but most systems still solve only one slice of the problem. Schäfer et al. evaluated LLMs for automated unit test generation and emphasized a central motivation that remains true today: unit tests are important, but writing them manually is laborious [8]. Meta’s TestGen-LLM moved the field closer to industrial use by automatically improving existing tests while filtering outputs for quality. Their reported results show why verification matters: generated tests may compile, run, and increase coverage at different rates, so automatic generation alone is not enough [9].

On the multi-agent side, ChatDev demonstrated that specialized agents can cooperate across design, coding, and testing through structured communication [10]. Fuzz4All showed that LLM-driven systems can also be highly effective in fuzzing-oriented testing, describing itself as a universal fuzzer and reporting 98 discovered bugs across nine

systems under test [11]. These projects are important because they prove three things: specialization matters, automation can increase reach, and verification must accompany generation. However, they do not directly solve the exact workflow target of AI Test Lab, which is to help a developer quickly move from “something may be wrong in this repository” to “here is the likely issue, the rationale, a candidate fix, a regression test, and a safe path toward a pull request.”

Positioning of AI Test Lab

AI Test Lab is not positioned as a replacement for pytest, CI, static analysis, or end-to-end testing. Instead, it acts as an orchestration layer that tries to reduce the manual work between defect suspicion and actionable engineering output.

5. The AI Test Lab / MOSAIC Approach

The system implemented in this project uses a research-style pipeline called MOSAIC: *Multi-agent Orchestrated Software Analysis with Inter-agent Consensus*. The key design idea is that different defect classes are easier to surface when different specialists look at the same code from different perspectives. Instead of relying on a single broad prompt, AI Test Lab assigns specialized roles for unit logic, black-box behavior, edge cases, security, white-box path analysis, and performance. These are followed by a verifier-like critic and a consensus stage before the interface allows PR-ready actions.

Figure 1. High-level workflow of AI Test Lab

1. Repository intake

GitHub authentication, repository selection, branch context, source retrieval.

2. Specialist analysis

Six specialist agents inspect correctness, behavior, edge cases, security, paths, and performance.

3. Verifier + consensus

Claims are clustered, challenged, confirmed or rejected, and scored for confidence.

4. Action packaging

Top issue summary, generated tests, fix pack, PR preview, and AI chat for refinement.

The system emphasizes explainability and safety. A pull request is only enabled when the issue is strong enough and a safe concrete patch exists.

5.1 Specialist-agent design

The current implementation uses six specialist agents:

- **Unit agent:** inspects function-level correctness and likely regressions.
- **Black-box agent:** reasons about inputs and outputs without relying only on implementation details.
- **Edge-case agent:** focuses on boundary conditions, empty inputs, unexpected states, and unusual paths.
- **Security agent:** searches for risky patterns such as unsafe execution, weak parsing, or insecure handling.
- **White-box agent:** inspects paths, branches, and control-flow behavior.
- **Performance agent:** flags inefficient structures and scaling risks.

After those agents produce claims, a critic/verifier checks whether a finding is plausible, confirmed, unverified, or rejected. A consensus stage then resolves whether the claim should survive into the final report. This matters because AI assistance is most useful when it explains why a finding deserves trust, not only when it produces more text.

5.2 Why the pull request can be unavailable

A core design decision in AI Test Lab is that automatic pull request creation is safety-gated. A finding may appear in the report and still remain “preview only” if the evidence is not strong enough or if a safe concrete patch was not generated. This is an intentional product choice. It is better for the system to say “I found a likely issue, but I am not confident enough to open a PR automatically” than to produce a misleading or risky branch.

In the current prototype, a PR-ready action requires four conditions:

1. The issue must survive the specialist-agent consensus step.
2. The verifier must support it strongly enough.

- 3. The confidence score must pass the threshold.
- 4. A concrete patch must be available, not just a review summary.

This gating mechanism is one of the main ways the system attempts to be safer than naive “auto-fix” experiences.

6. Why This Approach Can Be Better

AI Test Lab is better than a fragmented testing workflow in three practical ways. First, it reduces interpretation cost. Instead of forcing the user to combine CI output, scanner results, and manual notes, it attempts to surface the likely top issue directly. Second, it reduces mode-switching. The same run can produce an explanation, a fix idea, a regression test, and a PR preview. Third, it uses specialization and gating. The multi-agent design broadens the search space, while the verifier and confidence policy narrow what becomes actionable.

Importantly, “better” should be defined carefully. The project does not claim to outperform every conventional testing tool at every testing task. Rather, its advantage lies in workflow compression. It shortens the path from issue discovery to issue understanding and from issue understanding to proposed change. For student teams and fast-moving software teams, that kind of compression can be more valuable than any single gain in raw coverage or raw detection count.

Task	Traditional workflow	AI Test Lab workflow
Understand the likely failure	Read logs, inspect files, infer likely root cause	Top issue is summarized directly in the report
Look across different defect types	Use separate tools or mental passes for logic, security, and edge cases	Specialist agents analyze these perspectives in parallel
Decide whether the result is trustworthy	Manual judgment plus secondary tools	Verifier and consensus stages provide structured trust signals
Write regression tests	Developer writes tests from scratch after diagnosis	System can propose custom regression tests in the same report

Task	Traditional workflow	AI Test Lab workflow
Prepare code change communication	Manual pull request writing and explanation	Draft PR preview is prepared when the patch is safe enough

Table 2. The main time-saving value of AI Test Lab is not a claim that testing disappears; it is that diagnosis, explanation, regression authoring, and PR preparation happen in one flow.

6.1 Where the time savings come from

The time-saving claim of this project should be interpreted as a workflow claim, not an unverified percentage claim. Based on the implemented feature set, AI Test Lab can save time in the following ways:

- **Less context switching:** the user does not need to move between separate tools just to form a first diagnosis.
- **Faster first-pass triage:** the report starts with the likely main issue instead of leaving the user to read every raw output stream.
- **Reduced writing overhead:** candidate tests, fixes, and PR text are generated from the same defect context.
- **Better prioritization:** the verifier and confidence score help distinguish stronger findings from weaker suspicions.
- **Safer automation:** preview-only gating prevents the system from auto-opening a PR when confidence is low or no patch exists.

For academic honesty, this should be framed as *expected developer time savings in early defect triage and remediation preparation*, not as a universal replacement for full validation.

6.2 Illustrative benchmark comparison: single-pass baseline vs MOSAIC

To make the comparison more concrete, the current prototype includes a small Python benchmark corpus with four curated cases: three true-defect cases and one low-signal case. The same corpus can be processed in two ways. The first is the project’s simpler single-pass baseline mode, which resembles an ordinary one-shot review pass. The second is the MOSAIC multi-agent mode, which uses specialist agents, a verifier, and consensus. This comparison is useful because it shows not only whether the tool notices a problem, but also whether it suppresses weak findings and whether it produces deeper evidence around the same issue.

Benchmark case	Expected defect labels	Single-pass baseline	MOSAIC result	Interpretation
Bare-except integer parser	bare_except	1 finding surfaced	1 confirmed finding	Both modes detect the issue, but MOSAIC confirms it through three specialist perspectives.
Unsafe eval handler	dynamic_execution	1 finding surfaced	1 confirmed finding	Both modes identify the issue, but MOSAIC strengthens trust by combining security, black-box, and white-box evidence.
Low-signal utility module	none expected	1 weak fallback finding surfaced	0 consensus findings	MOSAIC avoids a low-signal false positive that the baseline still reports.

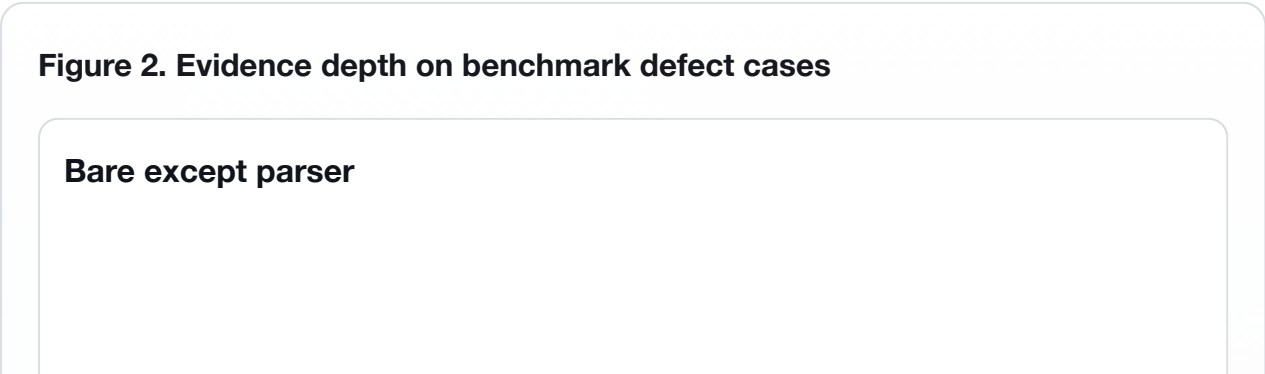
Benchmark case	Expected defect labels	Single-pass baseline	MOSAIC result	Interpretation
Zero-variance analytics helper	zero_variance_outlier	1 finding surfaced, PR-ready	1 confirmed finding, PR-ready	Both modes reach a useful action, but MOSAIC provides much richer justification for why the fix is safe enough.

Table 3. Results from the current four-case benchmark corpus on April 23, 2026. In this corpus, the main gain from MOSAIC is improved precision and stronger explanation depth rather than a larger raw count of surfaced defects.

Across the four benchmark cases, the single-pass baseline surfaced four findings in total, including one low-signal finding on a case where no defect was expected. MOSAIC surfaced three consensus findings, all aligned with the three true-defect cases, and suppressed the low-signal case. In other words, the benchmark suggests that MOSAIC is not merely “finding more things”; it is finding more trustworthy things and filtering weak signals more effectively.

6.3 Evidence depth: why MOSAIC can feel stronger than a normal first-pass test review

A user may reasonably ask: if both modes can still report the same top defect, why is MOSAIC better? The answer is evidence depth. In a simple first-pass workflow, a developer often receives one output stream and one conclusion. In MOSAIC, the same underlying defect can be supported by multiple specialists, followed by verifier review and consensus resolution. This produces a richer chain of trust before the interface proposes code changes.



Single-pass baseline: 1 surfaced issue.

MOSAIC: 3 supporting agents + verifier + consensus = 5 evidence layers.

Unsafe eval handler

Single-pass baseline: 1 surfaced issue.

MOSAIC: 3 supporting agents + verifier + consensus = 5 evidence layers.

Zero-variance metrics

Single-pass baseline: 1 surfaced issue.

MOSAIC: 4 supporting agents + verifier + consensus = 6 evidence layers.

Low-signal utilities

Single-pass baseline: 1 weak fallback issue.

MOSAIC: 0 consensus findings, which prevents unnecessary escalation.

Figure 2 shows why a multi-agent system can save time even when raw defect counts are similar: it reduces uncertainty, surfaces supporting rationale, and filters weak findings before the user writes tests or code changes.

This distinction is especially important for project demos and real engineering work. A baseline tool that finds “one problem” still leaves much of the reasoning burden on the developer. MOSAIC distributes that burden across specialist agents, then recombines the output into a structured decision. In practical terms, that means less time spent asking “is this really a bug?” and more time spent deciding “how should we fix and test it?”

7. Why AI Test Lab Is Distinct from Prior Work

Prior research has already shown that LLMs can generate tests, improve tests, coordinate multi-agent software tasks, and even fuzz complex systems. The distinctive contribution of AI Test Lab is the integration of these ideas into a single developer-facing workflow. It does not only generate tests. It does not only orchestrate agents. It does not only fuzz or review. Instead, it combines repository analysis, specialist perspectives, verifier-backed

consensus, human-readable diagnosis, generated test artifacts, proposed code changes, and PR packaging in one product loop.

This integration focus is especially useful for educational and practical engineering settings. A professor or developer can understand the story of the tool in one sentence: *AI Test Lab tries to tell you what is wrong, why it matters, what test should exist, how it could be fixed, and whether it is safe to open a PR.* That end-to-end narrative is the project's strongest advantage.

8. Limitations and Honest Boundaries

A research-style evaluation should also state the limitations clearly.

1. **The current prototype is strongest at AI-guided analysis and packaging, not at replacing native project execution.** For many repositories, the system currently behaves as an intelligent multi-perspective analysis layer rather than a full substitute for running the repository's own complete test suite.
2. **Automatic PR creation is intentionally conservative.** This improves safety, but also means users may sometimes see a useful report with no PR-ready action.
3. **The benchmark scope is still limited.** The present prototype is Python-first and uses a curated benchmark harness. Broader language support and larger public evaluations would strengthen the research story.
4. **Confidence scoring is helpful but not equivalent to ground truth.** It improves prioritization, but final engineering judgment remains necessary.

These limitations are not weaknesses to hide; they are the basis for future work. The next major step would be integrating true repository-native test execution, real failure output capture, and richer coverage evidence into the same MOSAIC workflow.

9. Conclusion

Software testing remains essential because it reduces delivery risk and protects software quality. Modern teams already use many powerful testing practices, including unit testing, CI, code scanning, security testing, and end-to-end validation. The problem is not that testing is absent. The problem is that the evidence produced by those practices is often scattered and expensive to turn into action.

AI Test Lab addresses that workflow problem by combining specialist-agent analysis, verifier-backed consensus, generated fixes and tests, and draft PR preparation inside a

single interface. Its MOSAIC pipeline is especially valuable because it uses different agents for different defect perspectives, then narrows actionability through confidence and verifier gating. That design makes the system more explainable and safer than a one-shot “auto-fix” approach. The most defensible claim for this project is therefore simple: AI Test Lab can save developer time by reducing context switching and shortening the path from defect suspicion to actionable engineering output.

In academic terms, the project demonstrates a credible direction for AI-assisted testing systems: not just generating tests, but orchestrating specialized reasoning, verification, explainability, and delivery artifacts in one coherent loop.

References

1. ISO. *ISO/IEC/IEEE 29119-2:2013 Software and systems engineering — Software testing — Part 2: Test processes*. ISO standard page. Available at: <https://www.iso.org/standard/56736.html>
2. Per Runeson. “A Survey of Unit Testing Practices.” *IEEE Software*, 23(4), 2006. Lund University entry: <https://portal.research.lu.se/en/publications/a-survey-of-unit-testing-practices/>
3. pytest documentation. “Full pytest documentation.” <https://docs.pytest.org/en/stable/contents.html>
4. GitHub Docs. “Continuous integration.” <https://docs.github.com/en/actions/get-started/continuous-integration>
5. GitHub Docs. “About code scanning.” <https://docs.github.com/en/code-security/concepts/code-scanning/about-code-scanning>
6. OWASP Developer Guide. “Security Testing activity / Verification tools overview.” <https://devguide.owasp.org/en/06-verification/02-tools/>
7. Cypress Documentation. “Testing Types.” <https://docs.cypress.io/app/core-concepts/testing-types>
8. Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. “An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation.” arXiv:2302.06527. <https://arxiv.org/abs/2302.06527>
9. Nadia Alshahwan et al. “Automated Unit Test Improvement using Large Language Models at Meta.” FSE 2024 Industry Paper / UCL Discovery open version. <https://discovery.ucl.ac.uk/id/eprint/10199775/>
10. Chen Qian et al. “ChatDev: Communicative Agents for Software Development.” arXiv:2307.07924. <https://arxiv.org/abs/2307.07924>

11. Chunqiu Steven Xia et al. “Fuzz4All: Universal Fuzzing with Large Language Models.”

Project page and paper links. <https://fuzz4all.github.io/>

Authoring note: this document explains the current AI Test Lab prototype honestly. It argues that the system improves the testing workflow, especially issue triage and remediation preparation, while acknowledging that full native test execution integration remains future work.